ExamWise: Intelligent Exam Question Difficulty Analysis System

Milestone 1 — Difficulty Analysis & Baseline System Setup

Project Report

Team EPOCH PIONEERS

Department of Computer Science and Engineering

February 23, 2026

Abstract

Traditional examination evaluation methods determine question difficulty only after students have attempted the exam, limiting the ability of educators to design balanced assessments in advance. This project presents **ExamWise**, a machine learning–based system that analyzes exam-style questions and predicts their difficulty level (*Easy*, *Medium*, *Hard*) using textual features derived from natural language processing techniques. In Milestone 1, we establish the complete data pipeline—from raw Stack Overflow question–answer data to a trained Logistic Regression classifier—achieving an overall accuracy of 65.71%. We also build analytical visualizations and a Streamlit-based web interface for interactive exploration. This report documents the methodology, implementation, results, and future directions of the project.

Contents

1 Introduction **3**
- 1.1 Problem Statement . . . 3
- 1.2 Scope of Milestone 1 . . . 3

2 Literature Review **3**

3 Dataset **4**
- 3.1 Data Source . . . 4
- 3.2 Data Reduction . . . 4
- 3.3 Difficulty Labelling . . . 4
- 3.4 Label Distribution . . . 4

4 Methodology **5**
- 4.1 Feature Engineering . . . 5
- 4.2 Text Preprocessing . . . 5
- 4.3 Text Vectorisation . . . 5
- 4.4 Classification Model . . . 6
- 4.5 Train–Test Split . . . 6

5 Results **6**
5.1 Model Performance 6
5.2 Visualisations 7
5.3 Hardest Questions Analysis 8

6 System Architecture **8**
6.1 Streamlit Application 9
6.2 Model Persistence 9

7 Discussion **9**
7.1 Strengths 9
7.2 Limitations 9

8 Future Work **9**

9 Conclusion **10**

A Project Structure **11**

B Team Contributions **11**

1 Introduction

In conventional educational assessment, question difficulty is understood only retrospectively—after students take the exam and scores are aggregated [1]. This reactive paradigm limits instructors who wish to construct well-balanced question papers *before* administration.

ExamWise addresses this gap by building a predictive system that estimates question difficulty from the question text itself, without requiring prior student performance data. The system leverages supervised machine learning and natural language processing (NLP) to classify questions into three difficulty tiers.

1.1 Problem Statement

Given a corpus of questions with associated community response behaviour, the objective is to:

1. Derive difficulty labels from historical engagement metrics.
2. Train a text-based classifier that generalises to unseen questions.
3. Provide analytical dashboards for question-level insights.

1.2 Scope of Milestone 1

Milestone 1 delivers the foundational components:

- Data acquisition, reduction, and labelling pipeline.
- Feature extraction using Bag-of-Words vectorisation.
- Baseline Logistic Regression model for multi-class classification.
- Analytical visualisations (difficulty distribution, accuracy analysis, hardest questions).
- Streamlit web application for end-to-end interaction.

2 Literature Review

Automated question difficulty estimation has been explored through several approaches in educational data mining:

- **Item Response Theory (IRT):** Classical psychometric models estimate item difficulty as a latent parameter from response patterns [2]. However, IRT requires observed response data and cannot predict difficulty for new items.

- **Text-based approaches:** Recent work uses NLP features (readability scores, syntactic complexity, word frequency) to predict difficulty from question text alone [3]. TF–IDF and word embeddings are commonly used representations.

- **Community Q&A platforms:** Stack Overflow data has been used as a proxy for educational content, where answer scores and response rates serve as engagement-based difficulty indicators [4].

Our approach combines community engagement metrics for label derivation with text-based features for classification, enabling prediction on unseen questions.

3 Dataset

3.1 Data Source

We use the **StackSample** dataset from Kaggle [5], which contains a 10% sample of Stack Overflow questions and answers. The raw dataset comprises:

Table 1: Raw dataset statistics

File	Original Rows	Reduced Rows
`Questions.csv`	∼19.9M	15,000
`Answers.csv`	∼17.7M	15,000
`Tags.csv`	∼1.9M	15,000

3.2 Data Reduction

Due to computational constraints, the dataset is reduced to 15,000 rows per file using `data_reduction.py`. The reduction preserves referential integrity by:

1. Selecting the first 15,000 questions.
2. Filtering answers whose `ParentId` matches the selected question IDs.
3. Filtering tags corresponding to those question IDs.

3.3 Difficulty Labelling

Since real student response data is unavailable, we use community engagement as a proxy. The **difficulty score** for each question is computed as:

$$\text{difficulty_score} = 1 - \frac{\text{max_answer_score}}{\text{max_answer_score} + \text{answer_count} + 1} \tag{1}$$

The intuition is that questions with high-scoring answers relative to their answer count are "easier" (the community converged on a good solution), while questions with low-scoring or few answers are "harder".

After min–max normalisation to $[0, 1]$, labels are assigned using fixed thresholds:

$$\text{label} = \begin{cases} \textit{Easy} & \text{if difficulty_score} < 0.33 \\ \textit{Medium} & \text{if } 0.33 \leq \text{difficulty_score} < 0.66 \\ \textit{Hard} & \text{if difficulty_score} \geq 0.66 \end{cases} \tag{2}$$

3.4 Label Distribution

The resulting distribution is highly imbalanced (Table 2):

Table 2: Difficulty label distribution in the processed dataset

Label	Count	Percentage
Hard	11,280	75.2%
Medium	2,192	14.6%
Easy	1,528	10.2%
Total	**15,000**	**100%**

4 Methodology

Figure 1 illustrates the end-to-end pipeline.

$$\texttt{Raw CSVs} \xrightarrow{\text{Reduce}} \texttt{15k subset} \xrightarrow{\text{Feature Eng.}} \texttt{Labelled Data}$$
$$\xrightarrow{\text{Preprocess}} \texttt{Clean Text} \xrightarrow{\text{Vectorise}} \texttt{BoW Matrix} \xrightarrow{\text{Train}} \texttt{LR Model}$$
$$\xrightarrow{\text{Evaluate}} \texttt{Metrics} \xrightarrow{\text{Visualise}} \texttt{Dashboard}$$

Figure 1: ExamWise Milestone 1 processing pipeline

4.1 Feature Engineering

For each question, the following features are computed from the answer data:

Table 3: Engineered features per question

Feature	Description
`max_answer_score`	Highest score among all answers
`avg_answer_score`	Mean score across all answers
`answer_score_variance`	Variance of answer scores
`answer_count`	Number of answers received
`question_score`	Community score of the question
`ratio`	$\frac{\text{max_answer_score}}{\text{answer_count}+1}$

4.2 Text Preprocessing

The question body undergoes the following transformations:

1. **HTML stripping:** Removal of HTML tags using regex `<[^>]*>`.
2. **Non-alphabetic removal:** Retains only alphabetical characters.
3. **Lowercasing:** Uniform case normalisation.
4. **Stop word removal:** English stop words excluded during vectorisation.

4.3 Text Vectorisation

The cleaned text is transformed into a numerical feature matrix using `CountVectorizer` (Bag-of-Words) from scikit-learn [6], with English stop words removed. Each question is represented as a sparse vector of word counts.

4.4 Classification Model

A **Logistic Regression** classifier is trained on the vectorised text features for multi-class difficulty prediction. The model uses the one-vs-rest (OvR) scheme:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x} + b_k}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x} + b_j}} \tag{3}$$

where $\mathbf{x}$ is the BoW feature vector, $\mathbf{w}_k$ and b_k are the weight and bias for class k, and $K = 3$ (Easy, Medium, Hard).

4.5 Train–Test Split

The dataset is split into 70% training and 30% testing:

- Training set: 10,500 samples
- Test set: 4,500 samples

5 Results

5.1 Model Performance

Table 4: Classification report on the test set ($n = 4{,}500$)

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Easy	0.14	0.10	0.12	473
Medium	0.20	0.15	0.17	643
Hard	0.76	0.83	0.80	3,384
Accuracy		**0.6571** (65.71%)		
Macro Avg	0.37	0.36	0.36	4,500
Weighted Avg	0.62	0.66	0.64	4,500

Key observations:

- The model achieves 65.71% overall accuracy, driven largely by correct predictions on the majority class (*Hard*).
- The *Easy* class has an F1-score of only 0.12, indicating the model struggles severely with minority classes.
- The macro-averaged F1 of 0.36 reveals that per-class performance is not balanced.
- The class imbalance (75% *Hard*) is the primary cause of poor minority-class performance.

5.2 Visualisations

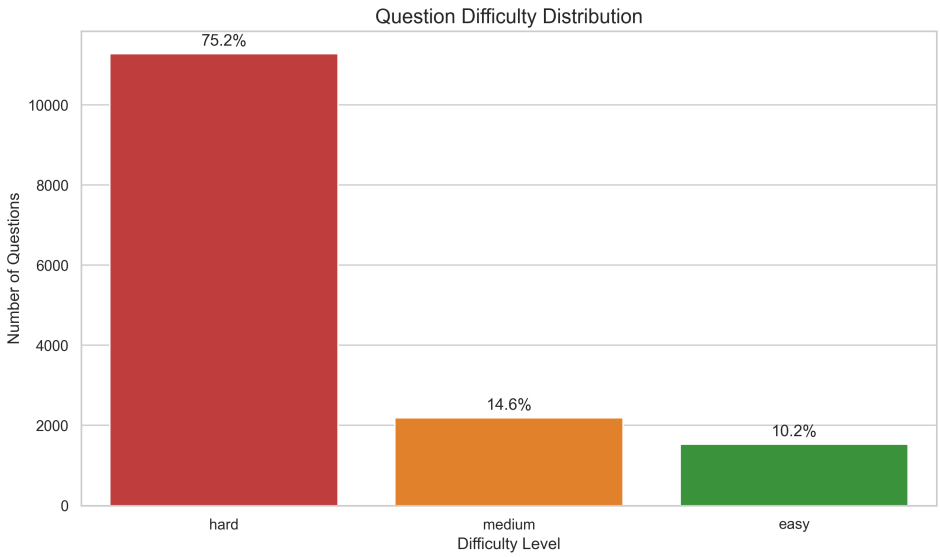

Figure 2: Distribution of difficulty labels across the processed dataset. The dataset is heavily skewed towards *Hard* questions (75.2%).

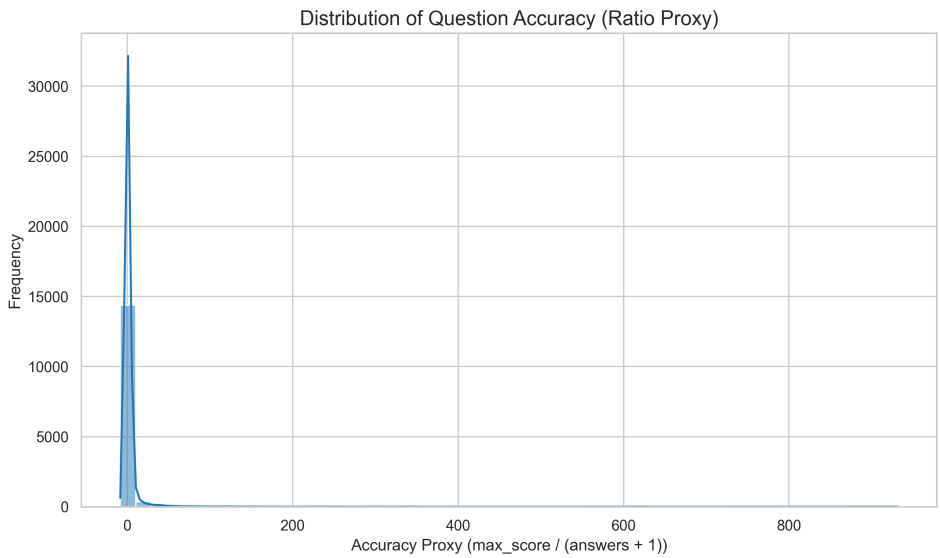

Figure 3: Distribution of the accuracy proxy metric (ratio $= \frac{\text{max_answer_score}}{\text{answer_count}+1}$). The distribution is right-skewed, with most questions concentrated near zero.

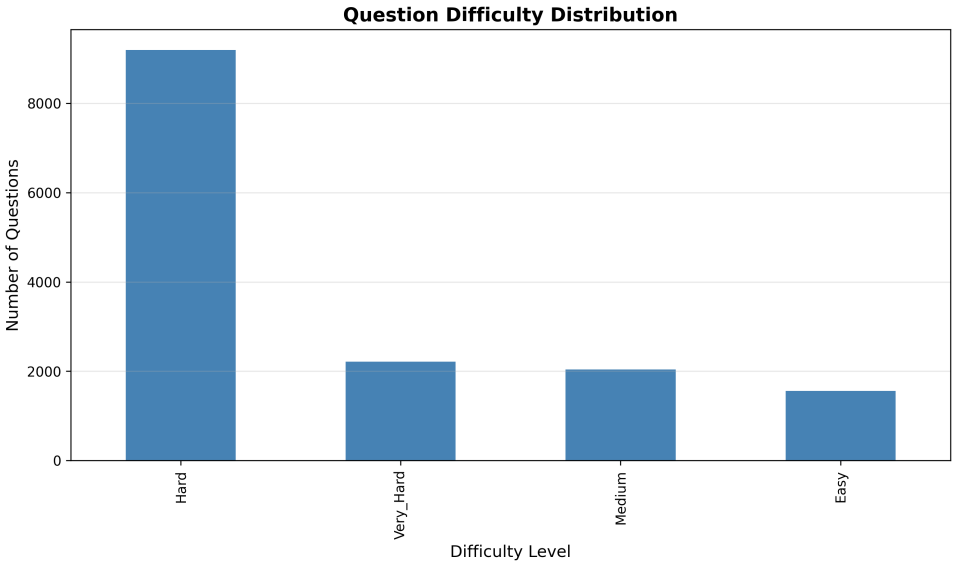

Figure 4: Bar chart showing the distribution of questions across difficulty categories.

5.3 Hardest Questions Analysis

Table 5 presents a sample of the most difficult questions identified by the system (lowest accuracy scores).

Table 5: Top 5 hardest questions (lowest accuracy proxy score)

ID	Title (Truncated)	Score
688245	Is there a better Python bundle for TextMate...	−17.0
183042	How can I use UUIDs in SQLAlchemy?	−15.0
688766	Getting 401 on Twitter OAuth POST requests	−1.5
284115	Cross platform hidden file detection	−1.0
336753	Django FormWizard and Admin application	−1.0

6 System Architecture

The system is organised into modular components:

Table 6: System modules and their responsibilities

Module	File	Responsibility
Data Reduction	`scripts/data_reduction.py`	Subsample raw CSVs
Feature Eng.	`scripts/feature_engineering.py`	Compute metrics and labels
Model Training	`notebooks/model_training.ipynb`	Train and evaluate classifier
Web Application	`app.py`	Streamlit UI
Visualisations	`outputs/*.png`	Static plots

6.1 Streamlit Application

The web interface (`app.py`) provides:

1. **CSV Upload:** Users upload `Questions.csv`, `Answers.csv`, and `Tags.csv`.
2. **Pipeline Execution:** One-click data reduction, feature engineering, and prediction.
3. **Results Display:** Table of true vs. predicted difficulty, performance metrics, and charts.

6.2 Model Persistence

The trained model and vectoriser are serialised using `joblib` [7]:

- `models/model.joblib` — Logistic Regression classifier (758 KB)
- `models/vectorizer.joblib` — CountVectorizer (447 KB)
- `models/metrics.txt` — Classification report

7 Discussion

7.1 Strengths

- End-to-end pipeline from raw data to interactive predictions.
- Modular code design with reusable scripts.
- Clear difficulty labelling formula grounded in engagement metrics.
- Interactive Streamlit interface enabling non-technical users to explore results.

7.2 Limitations

- **Class imbalance:** 75% of questions are labelled *Hard*, causing the model to be biased towards the majority class. Techniques such as SMOTE, class weighting (`class_weight='balanced'` in scikit-learn), or threshold tuning should be explored.
- **Single model:** Only Logistic Regression was trained. Comparing with Decision Trees, Random Forests, or gradient-boosted models would provide a more robust baseline.
- **Vectorisation:** CountVectorizer captures word presence but ignores word importance. TF–IDF weighting would down-weight common terms and improve discriminative power.
- **Proxy data:** Stack Overflow engagement metrics are an imperfect proxy for actual exam difficulty. Future work should incorporate real student response data.

8 Future Work

1. **Milestone 2:** Integrate TF–IDF vectorisation and train multiple models (Decision Tree, Random Forest, SVM) with hyperparameter tuning.

2. **Class imbalance handling:** Apply SMOTE or adjust class weights.

3. **Advanced NLP:** Experiment with word embeddings (Word2Vec, GloVe) or transformer-based models (BERT) for richer text representations.
4. **Real exam data:** Collect and integrate actual student response data for ground-truth difficulty labels.
5. **Agentic assessment design:** Use the difficulty predictor as part of an agentic system that automatically generates balanced exam papers.

9 Conclusion

Milestone 1 of the ExamWise project successfully establishes a working pipeline for question difficulty prediction. Starting from raw Stack Overflow data, the system reduces, processes, and labels 15,000 questions using an engagement-based difficulty formula. A Logistic Regression classifier trained on Bag-of-Words features achieves 65.71% accuracy, with strong performance on the majority class but poor recall on minority classes due to data imbalance.

The accompanying Streamlit application provides an accessible interface for uploading data, running the pipeline, and visualising results. While the current baseline has clear limitations—single model, no imbalance correction, simple vectorisation—it provides a solid foundation for the more advanced techniques planned in subsequent milestones.

References

[1] F. M. Lord, *Applications of Item Response Theory to Practical Testing Problems*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1980.

[2] R. K. Hambleton, H. Swaminathan, and H. J. Rogers, *Fundamentals of Item Response Theory*. Newbury Park, CA: Sage Publications, 1991.

[3] L. A. Ha, V. Yaneva, P. Baldwin, and J. Mee, "Automated estimation of item difficulty for multiple-choice tests: An application of word embedding techniques," in *Proc. 14th Workshop on Innovative Use of NLP for Building Educational Applications*, 2019, pp. 370–382.

[4] A. Anderson, D. Huttenlocher, J. Kleinberg, and J. Leskovec, "Discovering value from community activity on focused question answering sites: A case study of Stack Overflow," in *Proc. 18th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2012, pp. 850–858.

[5] Stack Overflow, "StackSample: 10% of Stack Overflow Q&A," Kaggle, 2016. [Online]. Available: https://www.kaggle.com/datasets/stackoverflow/stacksample. [Accessed: Feb. 20, 2026].

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] Joblib Development Team, "Joblib: Running Python functions as pipeline jobs." [Online]. Available: https://joblib.readthedocs.io/. [Accessed: Feb. 20, 2026].

A Project Structure

```
ExamWise/
  app.py                          # Streamlit application
  requirements.txt                # Python dependencies
  scripts/
    data_reduction.py             # Dataset reduction
    feature_engineering.py        # Feature computation and
       labelling
  models/
    model.joblib                  # Trained LR classifier
    vectorizer.joblib             # Fitted CountVectorizer
    metrics.txt                   # Classification report
  notebooks/
    model_training.ipynb          # Training notebook
  outputs/
    difficulty_distribution.png   # Difficulty bar chart
    accuracy_distribution.png     # Accuracy histogram
    hardest_questions.csv         # Hardest questions table
  docs/
    MODEL_CARD.md                 # Model documentation
    problem_statement.md          # Problem definition
    system_architecture.md        # Architecture overview
  report/
    main.tex                      # This report (LaTeX)
    references.bib                # Bibliography
    figures/                      # Report figures
```

Listing 1: Repository file structure

B Team Contributions

Table 7: Role-wise work allocation for Milestone 1

Role	Contributions
Person 1 (Data & Labelling)	Dataset acquisition, data reduction script, feature engineering, difficulty labelling, processed CSV export
Person 2 (Features & Models)	Model training, CountVectorizer pipeline, Logistic Regression classifier, evaluation metrics, model serialisation
Person 3 (Analytics & Visualisations)	Difficulty distribution charts, accuracy analysis, hardest questions identification, output generation
Person 4 (App & Integration)	Streamlit application, README documentation, requirements specification, Git management